

Architectures and Training Paradigms of Large Language Models: A Survey of Design Principles, Optimization Strategies, and Open Challenges

Guru Goutham Kanchi

Independent Research

July 2026

Abstract—Large Language Models (LLMs) have become the dominant paradigm in natural language processing, driven by the Transformer architecture and its capacity to scale across data, parameters, and compute. This paper surveys the architectural foundations of modern LLMs, including self-attention mechanisms, positional encoding schemes, and normalization strategies, and examines the training pipeline that transforms raw text corpora into instruction-following systems. We discuss pretraining objectives, tokenization strategies, distributed training techniques such as data, tensor, and pipeline parallelism, and the role of mixture-of-experts architectures in decoupling parameter count from inference cost. We further analyze post-training alignment methods, including supervised fine-tuning and reinforcement learning from human feedback, and discuss emerging alternatives such as direct preference optimization. The paper concludes with an examination of open challenges, including scaling law limitations, context length extension, hallucination, and the computational sustainability of frontier-scale training runs. Our goal is to provide a structured reference for researchers and practitioners seeking a consolidated view of the design space underlying contemporary LLMs.

Index Terms—large language models, transformer, self-attention, pretraining, fine-tuning, reinforcement learning from human feedback, mixture of experts, distributed training

I. INTRODUCTION

The past several years have witnessed a rapid transformation in natural language processing (NLP), driven almost entirely by the emergence of Large Language Models (LLMs) built on the Transformer architecture [1]. Unlike earlier recurrent and convolutional approaches, which processed sequences step-by-step or through fixed-size local receptive fields, the Transformer’s self-attention mechanism allows every token in a sequence to directly attend to every other token, enabling far more effective parallelization during training and more expressive modeling of long-range dependencies.

This architectural shift, combined with the observation that model performance improves predictably with increases in parameter count, dataset size, and compute budget [2], has motivated a race toward ever-larger models. Systems such as GPT-3 [3], PaLM, LLaMA, and their successors have demonstrated that sufficiently large models trained on broad corpora acquire emergent capabilities, including few-shot learning, multi-step reasoning, and code generation, without being explicitly trained for these tasks.

However, scale alone does not fully explain the success of modern LLMs. Equally important are the engineering choices that make training such models tractable: efficient attention implementations, mixed-precision arithmetic, distributed parallelism strategies, and carefully designed training curricula. Furthermore, the raw pretrained model is typically not directly useful as a conversational assistant; a second phase of training, involving supervised fine-tuning and human feedback, is required to align model outputs with user intent and safety expectations [4].

A useful way to frame the field is to distinguish three eras of language modeling. The statistical era relied on n -gram frequency counts and smoothing techniques, which captured local co-occurrence statistics but could not generalize beyond short, memorized contexts. The neural era introduced dense word embeddings and recurrent architectures such as LSTMs, which improved generalization but suffered from sequential computation bottlenecks and difficulty modeling long-range dependencies due to vanishing gradients. The current Transformer era resolves both limitations simultaneously: attention provides direct, gradient-friendly paths between arbitrarily distant tokens, while the absence of sequential recurrence allows full parallelization across the sequence dimension during training. This combination is arguably the single architectural decision most

responsible for the scalability that underlies the modern LLM paradigm.

It is also worth distinguishing between the encoder-only, encoder-decoder, and decoder-only variants of the Transformer, since this distinction shapes both training objective and downstream use. Encoder-only models such as BERT [7] are trained with masked-language-modeling objectives and excel at producing contextual representations for classification and retrieval tasks, but are not naturally suited to open-ended generation. Encoder-decoder models, exemplified by T5, separate the representation-building and generation roles into two stacks connected by cross-attention, and are well suited to sequence-to-sequence tasks such as translation and summarization. Decoder-only models, the architecture underlying GPT-family systems and the majority of contemporary general-purpose LLMs, use a single stack with causal (unidirectional) self-attention, so that generation and representation-building occur within the same computation. The decoder-only design has become dominant for general-purpose assistants primarily because a single autoregressive objective scales cleanly to arbitrarily large web-scale corpora without requiring task-specific input corruption schemes.

This paper aims to synthesize these strands into a coherent overview. Section II reviews the core architectural components of the Transformer and the design variants adopted by modern LLMs. Section III discusses pretraining, including data curation, tokenization, and optimization. Section IV covers distributed training techniques that make large-scale training computationally feasible. Section V discusses post-training alignment. Section VI discusses evaluation and known limitations. Section VII discusses open challenges and future directions, and Section VIII concludes.

II. TRANSFORMER ARCHITECTURE

A. Self-Attention Mechanism

The foundational building block of the Transformer is the scaled dot-product attention mechanism. Given a sequence of token embeddings, three learned linear projections produce query (Q), key (K), and value (V) matrices. The attention output is computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

where d_k is the dimensionality of the key vectors. The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large in magnitude, which would otherwise push the softmax function into regions with vanishingly small gradients.

Multi-head attention extends this mechanism by computing several attention operations in parallel, each with its own learned projections, and concatenating the results before a final linear projection. This allows the model to jointly attend to information from different representation subspaces, capturing syntactic, semantic, and positional relationships simultaneously.

A well-known drawback of full self-attention is that its computational and memory cost scales quadratically with sequence length n , since the attention matrix QK^T has dimension $n \times n$. This quadratic scaling becomes the dominant cost as context windows grow into the tens or hundreds of thousands of tokens, and has motivated a substantial body of work on efficient attention. Flash Attention [8] addresses this cost not by changing the mathematical operation but by changing how it is computed: rather than materializing the full $n \times n$ attention matrix in high-bandwidth memory, it computes attention in blocks, fusing the softmax and matrix-multiplication steps so that intermediate results remain in fast on-chip memory. This yields substantial wall-clock speedups and memory savings without any change to the numerical result, and has become a near-universal component of modern LLM training and inference stacks.

Modern LLMs also frequently adopt grouped-query attention (GQA) or multi-query attention (MQA), in which multiple query heads share a smaller number of key and value projections. Because the memory footprint of autoregressive generation is dominated by the key-value (KV) cache, reducing the number of distinct key/value heads substantially lowers memory bandwidth requirements during inference, at a modest cost in modeling capacity, making GQA a common choice in production-oriented model families.

B. Positional Encoding

Because self-attention is permutation-invariant, Transformers require an explicit mechanism to inject positional information. The original Transformer used fixed sinusoidal positional encodings, but modern LLMs have largely converged on Rotary Positional Embeddings (RoPE) [5], which encode relative position by rotating query and key vectors in a manner that depends only on their relative distance rather than absolute position. This has proven particularly beneficial for extending context length beyond the range seen during training, since the relative encoding generalizes more gracefully than absolute schemes.

C. Normalization and Residual Connections

Each Transformer block contains residual connections around both the attention and feed-forward sublayers, allowing gradients to flow directly through the network and mitigating vanishing gradient problems in deep stacks. Layer normalization is applied to stabilize activations; most contemporary architectures apply normalization prior to each sublayer (pre-norm) rather than after (post-norm), as pre-norm configurations exhibit substantially more stable

training dynamics at large depth, at some cost to representational capacity per layer.

D. Feed-Forward Networks and Activation Functions

Interleaved with attention layers are position-wise feed-forward networks, typically consisting of an up-projection, a nonlinearity, and a down-projection. While early Transformers used ReLU activations, most modern LLMs use gated variants such as SwiGLU, which have been empirically shown to improve downstream performance for a fixed parameter budget.

E. Mixture-of-Experts Architectures

A significant architectural development is the adoption of Mixture-of-Experts (MoE) layers [9], in which the dense feed-forward sublayer is replaced by a bank of expert subnetworks, only a small subset of which are activated for any given token via a learned routing function. In a typical top- k routing scheme, a lightweight gating network produces a score for each expert given the current token representation, and only the k highest-scoring experts (commonly $k = 1$ or $k = 2$) process that token, with their outputs combined via a weighted sum. This allows total parameter count to scale substantially while keeping the computational cost of a forward pass roughly constant, decoupling model capacity from inference latency.

Training MoE models introduces distinctive challenges not present in dense architectures. Because the router is trained jointly with the experts via gradient descent, it can collapse onto a small subset of experts early in training, a failure mode in which the remaining experts receive too few tokens to train effectively and become increasingly irrelevant, further starving them of gradient signal in a self-reinforcing cycle. To counteract this, MoE training typically incorporates an auxiliary load-balancing loss that penalizes uneven token distribution across experts, alongside capacity constraints that cap the number of tokens each expert may process per batch, with overflow tokens either dropped or passed through a residual path. Communication overhead is also a first-class concern in MoE training, since tokens routed to experts residing on different devices must be exchanged via all-to-all communication, making network topology and expert placement important considerations in distributed MoE training.

Table 1 summarizes representative architectural choices across several well-known model families, illustrating how the design space discussed above is combined in practice.

F. Embedding and Output Layers

The input embedding matrix, which maps discrete token identifiers to continuous vectors, and the output projection matrix, which maps final hidden states back to a distribution over the vocabulary, are both of dimension proportional to vocabulary size times hidden dimension, and together can constitute a non-trivial fraction of total parameters for models with large vocabularies. Many architectures tie these two matrices together, using a single shared weight matrix (transposed) for both the input embedding and output projection. Weight tying reduces parameter count and has been found in some settings to improve performance, plausibly because it imposes a useful inductive bias that tokens which are semantically similar as inputs should also

Table 1: Representative Architectural Choices Across Model Families

Aspect	Dense De-coder	MoE De-coder	Notes
Position encoding	RoPE	RoPE	Near-universal in recent models
Attention	MQA/GQA	MQA/GQA	Reduces KV-cache size
Activation	SwiGLU	SwiGLU	Improves quality per parameter
Normalization	Pre-norm RMSNorm	Pre-norm RMSNorm	Stabilizes deep stacks
FFN ex-pts	1 (dense)	8–64+	Only top- k active per token
Param. vs. compute	Coupled	Decoupled	MoE trades memory for compute

be similar as prediction targets, though very large models increasingly forgo tying in favor of independently sized input and output matrices when the resulting flexibility outweighs the parameter savings.

III. PRETRAINING

A. Data Curation and Tokenization

Pretraining corpora for modern LLMs are drawn from web crawls, books, code repositories, and curated reference sources, typically totaling trillions of tokens. Data quality filtering, deduplication, and toxicity screening are essential preprocessing steps, as raw web data is highly redundant and contains substantial low-quality or harmful content.

Text is converted into discrete tokens using subword tokenization algorithms, most commonly byte-pair encoding (BPE) or unigram language model tokenizers. Subword tokenization strikes a balance between the small, fixed vocabulary of character-level models and the large, sparse vocabulary of word-level models, allowing rare and out-of-vocabulary words to be represented as sequences of common subword units. BPE tokenizers are typically trained by starting from individual bytes or characters and iteratively merging the most frequent adjacent pair into a new symbol until a target vocabulary size, commonly in the range of 32,000 to 128,000 tokens for contemporary LLMs, is reached. Larger vocabularies reduce the average number of tokens required to represent a given piece of text, which lowers the effective sequence length and thus compute cost at a given context window, but larger vocabularies also enlarge the embedding and output projection matrices, so vocabulary size is itself a hyperparameter subject to trade-offs. Operating over raw UTF-8 bytes as the base alphabet, rather than a language-specific set of characters, additionally ensures that the tokenizer can represent arbitrary text, including code, symbols, and non-Latin scripts, without falling back to an out-of-vocabulary token.

B. Data Mixture and Curriculum

Beyond raw scale, the composition of the pretraining corpus materially affects downstream capability. Pretraining data is typically drawn from a mixture of sources, including filtered web crawls, books, encyclopedic text, source code, scientific articles, and dialogue data, combined according to sampling weights chosen to balance breadth of coverage

against the risk of over- or under-representing any single domain. Source code in particular has been observed to disproportionately benefit general-purpose reasoning ability, plausibly because code requires precise, compositional, multi-step logical structure. Some training pipelines additionally employ a data curriculum, in which the mixture of sources or the average difficulty of examples is varied over the course of training, for instance by increasing the proportion of high-quality curated data in the final portion of training after the majority of broad-coverage learning has taken place, a strategy sometimes referred to as an annealing or cooldown phase.

C. Training Objective

The dominant pretraining objective for generative LLMs is autoregressive next-token prediction: given a sequence of tokens $x_1, \dots, x_{t-1}$, the model is trained to maximize the likelihood of the true next token x_t . The overall training loss over a corpus is the average negative log-likelihood

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t | x_{<t}) \quad (2)$$

This objective is notable for its simplicity and scalability: it requires no labeled data and can be applied uniformly to any text corpus, which is a key reason large-scale pretraining has proven so effective.

D. Optimization

LLMs are typically trained using variants of the Adam optimizer with decoupled weight decay (AdamW), combined with a learning rate schedule that includes a warmup phase followed by cosine or linear decay. Gradient clipping is used to prevent instability from occasional large gradient norms. Mixed-precision training, using bfloat16 or float16 representations for most computations while maintaining a master copy of weights in higher precision, substantially reduces memory footprint and increases training throughput on modern accelerators.

IV. DISTRIBUTED TRAINING AT SCALE

Training frontier-scale LLMs requires distributing computation across thousands of accelerators, which necessitates combining several parallelism strategies.

A. Data Parallelism

In data parallelism, each device holds a full copy of the model and processes a different shard of the training batch; gradients are synchronized (typically via all-reduce operations) after each step. This is the simplest form of parallelism but is limited by the requirement that the full model fit within a single device’s memory.

B. Tensor and Pipeline Parallelism

Tensor parallelism [10] partitions individual weight matrices across devices, splitting matrix multiplications within a layer so that, for example, each device computes only a slice of the attention heads or a slice of the feed-forward hidden dimension before results are combined via a collective communication operation. Pipeline parallelism instead assigns different layers of the model to different devices and pipelines micro-batches through them to keep all

devices utilized; naive pipelining, however, leaves devices idle while waiting for activations to propagate through the pipeline (the so-called pipeline bubble), which schedulers mitigate by splitting batches into many small micro-batches and interleaving forward and backward passes across pipeline stages. Both tensor and pipeline parallelism allow models that exceed the memory capacity of a single device to be trained, at the cost of increased communication overhead, and are typically applied within a single high-bandwidth compute node (tensor parallelism) or across nodes (pipeline parallelism) to match the communication pattern to the available network bandwidth at each level of the hardware hierarchy.

C. ZeRO and Memory Optimization

The Zero Redundancy Optimizer (ZeRO) family of techniques [11] reduces memory redundancy in data-parallel training by partitioning optimizer states, gradients, and optionally parameters across devices rather than replicating them, enabling significantly larger models to be trained under a fixed memory budget per device. ZeRO is typically described in three progressive stages: Stage 1 partitions only the optimizer state (which, for Adam-family optimizers, requires two additional copies of each parameter for the first and second moment estimates and therefore dominates memory usage); Stage 2 additionally partitions gradients; and Stage 3 additionally partitions the model parameters themselves, gathering each parameter shard on demand immediately before it is needed for computation and releasing it immediately afterward. Stage 3 achieves the greatest memory savings but incurs the highest communication cost, since parameters must be repeatedly gathered across devices during both the forward and backward passes.

In practice, frontier training runs combine data, tensor, pipeline, and ZeRO-style parallelism into a single hybrid strategy (sometimes termed 3D or 4D parallelism) tuned to the specific hardware topology, typically favoring tensor parallelism within a node where interconnect bandwidth is highest, pipeline parallelism across nodes within a rack, and data parallelism at the outermost, highest-latency scale across racks or data centers.

V. POST-TRAINING ALIGNMENT

A. Supervised Fine-Tuning

Following pretraining, models are typically fine-tuned on a curated dataset of instruction-response pairs, teaching the model to follow natural language instructions and adopt a consistent conversational format. This stage is comparatively inexpensive relative to pretraining, since it requires far fewer tokens, but has an outsized effect on the model’s usability.

B. Reinforcement Learning from Human Feedback

Reinforcement Learning from Human Feedback (RLHF) further aligns model behavior with human preferences [4]. A reward model is trained on human-ranked comparisons between candidate responses, and the language model is then optimized against this reward signal, typically using Proximal Policy Optimization (PPO), while a Kullback-Leibler divergence penalty against the original supervised model discourages the policy from drifting too far and de-

generating into reward hacking.

C. Parameter-Efficient Fine-Tuning

Full fine-tuning of a large pretrained model, in which every parameter is updated, requires storing optimizer state proportional to the full parameter count and is often impractical outside of well-resourced settings. Low-Rank Adaptation (LoRA) [12] addresses this by freezing the pretrained weight matrices entirely and instead learning a pair of small low-rank matrices whose product is added to each frozen weight matrix as a correction term. Because the rank of this correction is chosen to be far smaller than the dimensions of the original matrix, the number of trainable parameters is reduced by orders of magnitude relative to full fine-tuning, while empirically achieving comparable downstream task performance for many adaptation scenarios. LoRA adapters can also be merged back into the base weights after training at no additional inference cost, or kept separate to allow several task-specific adapters to be swapped in and out of a single shared base model.

D. Direct Preference Optimization

Direct Preference Optimization (DPO) [6] has emerged as a simpler alternative to RLHF, reformulating the preference-alignment objective as a closed-form classification loss over pairs of preferred and dispreferred responses, avoiding the need to train a separate reward model or run reinforcement learning at all. Empirically, DPO achieves alignment quality comparable to PPO-based RLHF with substantially reduced implementation complexity and training instability.

VI. EVALUATION AND LIMITATIONS

A. Benchmark-Based Evaluation

Evaluating LLMs remains methodologically challenging. Static benchmark suites measuring knowledge, reasoning, and coding ability are widely used but are increasingly susceptible to contamination, wherein benchmark data leaks into pretraining corpora, inflating reported performance in a manner that does not reflect genuine generalization. Because pretraining corpora are drawn heavily from web text, and popular benchmarks are themselves hosted on the web, avoiding contamination requires active effort, such as canary strings or held-out benchmark variants, and even these mitigations are imperfect at web scale.

B. Preference-Based and Human Evaluation

Human and model-based preference evaluations, in which outputs are compared head-to-head and a preference is recorded, are increasingly used to complement static benchmarks, and are the basis for widely cited public leaderboards. These evaluations introduce their own biases, however, including a tendency for both human and automated judges to favor longer, more confidently phrased, or more elaborately formatted responses independent of actual correctness, a phenomenon sometimes termed length bias or style bias. When a model being evaluated is itself used to judge outputs (LLM-as-judge), further concerns arise regarding self-preference, in which a judge model rates outputs resembling its own style more favorably.

Table 2 lists representative benchmark categories com-

Table 2: Representative Evaluation Categories for LLMs

Category	What It Measures
Knowledge QA	Factual recall across broad academic and general-knowledge domains
Mathematical reasoning	Multi-step arithmetic and symbolic problem solving
Code generation	Correctness of generated programs against functional test cases
Long-context retrieval	Ability to locate and use information placed within a long input
Instruction following	Adherence to explicit constraints in the user’s request
Preference win-rate	Head-to-head human or model preference between responses

monly used to characterize LLM capability across different axes.

C. Hallucination

A persistent limitation of LLMs is hallucination: the generation of fluent, plausible-sounding but factually incorrect content. Because the pretraining objective rewards predicting plausible continuations rather than verified truth, models have no intrinsic mechanism for distinguishing memorized fact from confident fabrication, and post-training alignment only partially mitigates this behavior. Hallucination is particularly pronounced for long-tail factual queries involving rare entities, precise numerical claims, and citations, where the model may have seen insufficient or inconsistent training signal to reliably recall the correct answer, yet still produces a fluent and superficially plausible response due to the strength of the underlying language modeling capability. Retrieval-augmented generation, in which relevant external documents are retrieved and provided as context at inference time, is a widely used mitigation, though it introduces its own failure modes if retrieval quality is poor or if the model fails to appropriately prioritize retrieved evidence over its parametric knowledge.

VII. INFERENCE AND DEPLOYMENT CONSIDERATIONS

Training determines what a model can do, but serving determines whether that capability can be delivered at acceptable cost and latency, and a substantial body of engineering effort is devoted specifically to inference efficiency.

A. The KV Cache

During autoregressive generation, each newly generated token attends to all previous tokens’ key and value vectors. Recomputing these vectors from scratch at every generation step would be wastefully redundant, so practical implementations cache the key and value vectors for all previously processed tokens, the so-called KV cache, and reuse them at each subsequent step, computing only the query, key, and value for the newest token. The KV cache grows linearly with sequence length and batch size, and for long-context, high-throughput serving scenarios often becomes the dominant consumer of accelerator memory, which is precisely the motivation for memory-saving attention variants such as grouped-query and multi-query attention discussed in Section II.

B. Quantization

Quantization reduces the numerical precision used to store model weights and, in some schemes, activations, from a 16- or 32-bit floating point representation down to 8-bit integers or even lower. Because inference for large models is frequently memory-bandwidth bound rather than compute bound, reducing the number of bytes that must be moved from memory to the compute units for each parameter can yield substantial latency and throughput improvements with comparatively modest loss in output quality, particularly when quantization is applied carefully with per-channel or per-group scaling factors rather than a single global scale. Post-training quantization, applied after training completes, is far more common in practice than quantization-aware training, since it requires no additional training compute and can be applied directly to an already-trained checkpoint.

C. Speculative Decoding

Speculative decoding accelerates generation by using a small, fast draft model to propose several candidate tokens ahead of the current position, which the large target model then verifies in a single parallel forward pass; tokens that match what the target model would have generated are accepted, while the first mismatching token is corrected and subsequent draft tokens are discarded. Because verifying several tokens in one parallel forward pass is markedly more efficient on modern accelerators than generating them one at a time, speculative decoding can substantially reduce wall-clock generation latency without changing the output distribution of the target model, provided the acceptance/rejection procedure is implemented to preserve the target model’s exact sampling distribution.

D. Batching and Serving Systems

Because individual generation requests are memory-bandwidth bound rather than compute bound, serving systems achieve much higher hardware utilization by batching multiple concurrent requests together. Continuous batching, in which completed sequences are evicted and new requests are admitted into a running batch on a per-step basis rather than waiting for an entire batch to finish, substantially improves throughput relative to static batching in workloads with highly variable sequence lengths, which are the norm in conversational deployment.

VIII. OPEN CHALLENGES AND FUTURE DIRECTIONS

A. Scaling Law Limitations

While empirical scaling laws have reliably predicted performance gains from increased compute, evidence suggests diminishing returns as high-quality, non-redundant training data becomes scarce relative to model capacity. This has motivated growing interest in data quality over raw quantity, synthetic data generation, and multi-epoch training strategies.

B. Long-Context Modeling

Extending effective context length remains an active research area. The quadratic computational cost of full self-attention with respect to sequence length motivates sparse

attention patterns, sliding-window attention, and retrieval-augmented approaches that offload long-range memory to an external index rather than the attention mechanism itself.

C. Efficiency and Sustainability

The computational and energy cost of training and serving frontier LLMs raises sustainability concerns. Techniques such as quantization, knowledge distillation, and speculative decoding aim to reduce inference cost without proportionally sacrificing quality, and are increasingly important as LLMs are deployed at scale.

D. Reliability and Alignment

Ensuring that model behavior remains reliable, non-deceptive, and aligned with intended values across an ever-widening range of deployment contexts remains an open problem, particularly as models are granted greater autonomy through tool use and agentic workflows.

E. Multimodality and Tool Use

Contemporary LLMs are increasingly trained not purely on text but jointly on interleaved text, image, audio, and, in some cases, video data, typically by mapping non-text modalities into the same embedding space consumed by the Transformer backbone, either through a learned projection of pretrained encoder features or through direct tokenization of the raw modality. This extends the range of tasks a single model can perform but complicates data curation and evaluation, since quality signals and contamination risks differ substantially across modalities. Separately, the extension of LLMs into agentic systems, in which a model plans multi-step tasks, invokes external tools or APIs, and incorporates the results of those calls into subsequent reasoning, introduces new failure modes beyond those of single-turn text generation, including compounding errors across steps and the risk of executing consequential actions on the basis of a hallucinated intermediate conclusion. Robust agentic behavior is an active area of both architectural and training research, rather than a solved extension of existing pretraining and alignment recipes.

IX. CONCLUSION

The rapid progress of large language models has been driven by the interplay of a scalable architecture, the Transformer; a simple and data-efficient training objective, autoregressive next-token prediction; and substantial engineering investment in distributed training infrastructure spanning data, tensor, pipeline, and memory-partitioned parallelism. Post-training alignment techniques, from supervised fine-tuning through RLHF and its simplified successors such as DPO, have proven essential to converting raw pretrained capability into usable, instruction-following systems, while parameter-efficient methods such as LoRA have made adaptation to new tasks and domains accessible outside of the largest training facilities. Inference-side innovations, including quantization, speculative decoding, and efficient KV-cache management, have similarly proven necessary to make trained models practically deployable at acceptable cost and latency.

Looking forward, the trajectory of the field appears increasingly likely to be shaped less by indiscriminate increases in parameter count and more by a combination

of factors: the availability and quality of training data as high-quality text becomes a scarcer resource than raw compute; architectural refinements that improve the efficiency of long-context and multimodal reasoning; and alignment techniques capable of maintaining reliability as models are deployed with greater autonomy in agentic and tool-using settings. Taken together, these open problems suggest that the design space surveyed in this paper, spanning attention mechanisms, training curricula, distributed systems engineering, and post-training alignment, will remain an active and consequential area of research for the foreseeable future, rather than a solved engineering problem awaiting only additional scale.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
- [3] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [4] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [5] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.
- [6] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.
- [8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [9] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *arXiv preprint arXiv:1701.06538*, 2017.
- [10] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

- [11] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory optimizations toward training trillion parameter models,” in *Proc. SC20*, 2020.
- [12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.